

Object-Oriented Programming

LECTURER: NGUYEN THI THU TRANG, TRANGNTT@SOICT.HUST.EDU.VN

Lab 04: GUI Programming with Swing

In this lab, you will practice with:

- Create simple GUI applications with Swing
- Convert part of the Aims Project from the console/command-line (CLI) application to the GUI one.

0 Assignment Submission

For this lab class, you will have to turn in your work twice, specifically:

- **Right after the class:** for this deadline, you should include any work you have done within the lab class.
- **10 PM four days after the class:** for this deadline, you should include your work on all sections of this lab, and push it to a branch called “*release/lab04*” of the valid repository.

After completing all the exercises in the lab, you have to update the use case diagram and the class diagram of the AIMS project.

Each student is expected to turn in his or her work and not give or receive unpermitted aid. Otherwise, we would apply extreme methods for measurement to prevent cheating. Please write down answers for all questions into a text file named “**answers.txt**” and submit it within your repository.

1 Swing components

Note: For the exercises in this lab (excluding the AIMS exercises), you will create a new Java project named `GUIProject`, and put all your source code in a package called **"`hust.soict.dsai.swing`"** (for DS & AI).

Note: From this section onwards, it is assumed that you are a DS-AI student, so your folder structure will contain the **"`dsai`"** package. If you are an HEDSPI or ICT student, you should replace the **"`dsai`"** string with **"`hedspi`"** or **"`globalict`"**.

In this exercise, we revisit the elements of the Swing API and compare them with those of AWT by implementing the same mini-application using the two libraries. The application is an accumulator which accumulates the values entered by the user and displays the sum.

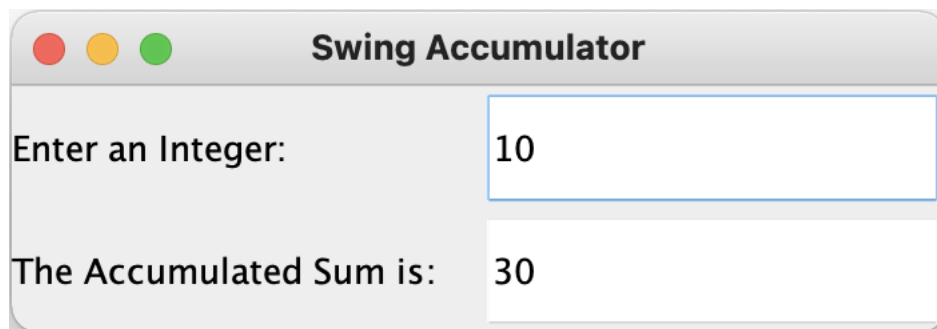


Figure 1. SwingAccumulator

1.1 AWTAccumulator

1.1.1 Create class AWTAccumulator with the source code as below

```
6 public class AWTAccumulator extends Frame {
7     private TextField tfInput;
8     private TextField tfOutput;
9     private int sum = 0;           // Accumulated sum, init to 0
10
11     // Constructor to setup the GUI components and event handlers
12     public AWTAccumulator() {
13         setLayout(new GridLayout(2, 2));
14
15         add(new Label("Enter an Integer: "));
16
17         tfInput = new TextField(10);
18         add(tfInput);
19         tfInput.addActionListener(new TFInputListener());
20
21         add(new Label("The Accumulated Sum is: "));
22
23         tfOutput = new TextField(10);
24         tfOutput.setEditable(false);
25         add(tfOutput);
26
27         setTitle("AWT Accumulator");
28         setSize(350, 120);
29         setVisible(true);
30     }
31
32     public static void main(String[] args) {
33         new AWTAccumulator();
34     }
35
36     private class TFInputListener implements ActionListener {
37         @Override
38         public void actionPerformed(ActionEvent evt) {
39             int numberIn = Integer.parseInt(tfInput.getText());
40             sum += numberIn;
41             tfInput.setText("");
42             tfOutput.setText(sum + "");
43         }
44     }
45 }
```

Figure 2. Source code of AWTAccumulator

1.1.2 Explanation

- In AWT, the top-level container is `Frame`, which is inherited by the application class.
- In the constructor, we set up the GUI components in the `Frame` object and the event-handling:
 - In line 13, the layout of the frame is set as `GridLayout`
 - In line 15, we add the first component to our `Frame`, an anonymous `Label`
 - In lines 17-19, we add a `TextField` component to our `Frame`, where the user will enter values. We add a listener which takes this `TextField` component as the source, using a named inner class.
 - In line 21, we add another anonymous `Label` to our `Frame`
 - In lines 23 – 25, we add a `TextField` component to our `Frame`, where the accumulated sum of entered values will be displayed. The component is set to read-only in line 24.
 - In line 27 – 29, the title & size of the `Frame` is set, and the `Frame` visibility is set to `true`, which shows the `Frame` to us.
- In the listener class (line 36 - 44), the `actionPerformed()` method is implemented, which handles the event when the user hits “Enter” on the source `TextField`.
 - In lines 39-42, the entered number is parsed, added to the sum, and the output `TextField`’s text is changed to reflect the new sum.
- In the `main()` method, we invoke the `AWTAccumulator` constructor to set up the GUI

1.2 SwingAccumulator

1.2.1 Create class SwingAccumulator with the source code as below:

```
8 public class SwingAccumulator extends JFrame {
9     private JTextField tfInput;
10    private JTextField tfOutput;
11    private int sum = 0;           // Accumulated sum, init to 0
12
13    // Constructor to setup the GUI components and event handlers
14    public SwingAccumulator() {
15        Container cp = getContentPane();
16        cp.setLayout(new GridLayout(2, 2));
17
18        cp.add(new JLabel("Enter an Integer: "));
19
20        tfInput = new JTextField(10);
21        cp.add(tfInput);
22        tfInput.addActionListener(new TFInputListener());
23
24        cp.add(new JLabel("The Accumulated Sum is: "));
25
26        tfOutput = new JTextField(10);
27        tfOutput.setEditable(false);
28        cp.add(tfOutput);
29
30        setTitle("Swing Accumulator");
31        setSize(350, 120);
32        setVisible(true);
33    }
34
35    public static void main(String[] args) {
36        new SwingAccumulator();
37    }
38
39    private class TFInputListener implements ActionListener {
40        @Override
41        public void actionPerformed(ActionEvent evt) {
42            int numberIn = Integer.parseInt(tfInput.getText());
43            sum += numberIn;
44            tfInput.setText("");
45            tfOutput.setText(sum + "");
46        }
47    }
48 }
```

Figure 3. Source code of SwingAccumulator

1.2.2 Explanation

- In Swing, the top-level container is JFrame which is inherited by the application class.
- In the constructor, we set up the GUI components in the JFrame object and the event-handling:

- Unlike AWT, the `JComponents` shall not be added onto the top-level container (e.g., `JFrame`, `JApplet`) directly because they are lightweight components. The `JComponents` must be added onto the so-called content-pane of the top-level container. Content-pane is in fact a `java.awt.Container` that can be used to group and layout components.
- In line 15, we get the content pane of the top-level container.
- In line 16, the layout of the content-pane is set as `GridLayout`
- In line 18, we add the first component to our content pane, an anonymous `JLabel`
- In lines 20-22, we add a `JTextField` component to our content-pane, where the user will enter values. We add a listener which takes this `JTextField` component as the source.
- In line 24, we add another anonymous `JLabel` to our content-pane
- In lines 26 – 28, we add a `JTextField` component to our content pane, where the accumulated sum of entered values will be displayed. The component is set to read-only in line 27.
- In line 30 – 32, the title & size of the `JFrame` is set, and the Frame visibility is set to true, which shows the `JFrame` to us.
- In the listener class (lines 39 - 47), the code for event-handling is exactly like the `AWTAccumulator`.
- In the `main()` method, we invoke the `SwingAccumulator` constructor to set up the GUI

1.3 Compare Swing and AWT elements:

Programming with AWT and Swing is quite similar (similar elements including container/components, and event-handling). However, there are some differences that you need to note:

- Compare the top-level containers in Swing and AWT
- Compare the class name of components in AWT and the corresponding class's name in Swing
- Compare the event-handling of Swing and AWT applications

The resulting appearances of the applications developed using Swing and AWT might be different as well. Make comparisons.

2 Organizing Swing components with Layout Managers

In Swing, there are two groups of GUI classes, the containers and the components. We have worked with several component classes in previous exercises. Now, we will investigate more on the containers.

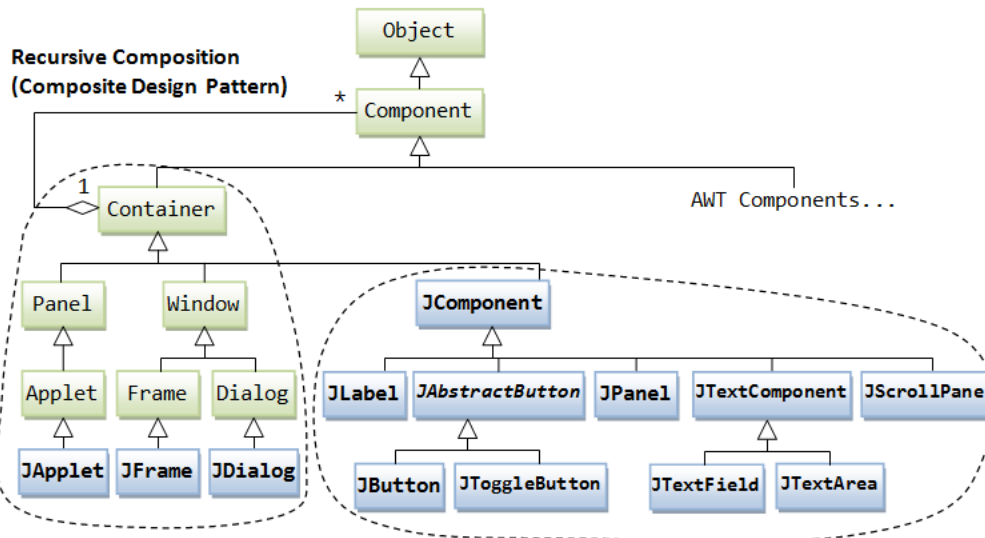


Figure 7. AWT and Swing elements

2.1 Swing top-level and secondary-level containers:

A container is used to hold components. A container can also hold containers because it is a (subclass of) component. Swing containers are divided into top-level and secondary-level containers:

- Top-level containers:
 - JFrame: used for the application's main window (with an icon, a title, minimize/maximize/close buttons, an optional menu bar, and a content pane)
 - JDialog: used for a secondary pop-up window (with a title, a close button, and a content pane).
 - JApplet: used for the applet's display area (content pane) inside a browser's window
- Secondary-level containers can be used to group and layout relevant components (most commonly used is JPanel)

2.2 Using JPanel as a secondary-level container to organize components:

2.2.1 Create class NumberGrid:

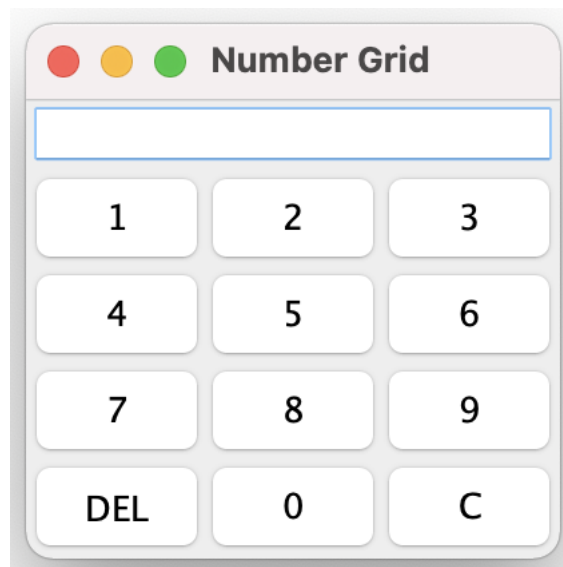


Figure 8. NumberGrid

This class allows us to input a number digit-by-digit from a number grid into a text field display. We can also delete the latest digit or delete the entire number and start over.

```

16 public class NumberGrid extends JFrame{
17     private JButton[] btnNumbers = new JButton[10];
18     private JButton btnDelete, btnReset;
19     private JTextField tfDisplay;
20
21     public NumberGrid() {
22
23         tfDisplay = new JTextField();
24         tfDisplay.setComponentOrientation(
25             ComponentOrientation.RIGHT_TO_LEFT);
26
27         JPanel panelButtons = new JPanel(new GridLayout(4, 3));
28         addButtons(panelButtons);
29
30         Container cp = getContentPane();
31         cp.setLayout(new BorderLayout());
32         cp.add(tfDisplay, BorderLayout.NORTH);
33         cp.add(panelButtons, BorderLayout.CENTER);
34
35         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
36         setTitle("Number Grid");
37         setSize(200, 200);
38         setVisible(true);
39     }

```

Figure 9. NumberGrid source code(1)

The class has several attributes:

- The `btnNumbers` array for the digit buttons
- The `btnDelete` for the DEL button
- The `btnReset` for the C button
- The `tfDisplay` for the top display

In the constructor, we add two components to the content pane of the `JFrame`:

- A `JTextField` for the display text field
- A `JPanel`, which will group all of the buttons and put them in a grid layout

2.2.2 Adding buttons

We add the buttons to the `panelButtons` in the `addButtons()` method:

```

40- void addButtons(JPanel panelButtons) {
41     ButtonListener btnListener = new ButtonListener();
42     for(int i = 1; i <= 9; i++) {
43         btnNumbers[i] = new JButton(""+i);
44         panelButtons.add(btnNumbers[i]);
45         btnNumbers[i].addActionListener(btnListener);
46     }
47
48     btnDelete = new JButton("DEL");
49     panelButtons.add(btnDelete);
50     btnDelete.addActionListener(btnListener);
51
52     btnNumbers[0] = new JButton("0");
53     panelButtons.add(btnNumbers[0]);
54     btnNumbers[0].addActionListener(btnListener);
55
56     btnReset = new JButton("C");
57     panelButtons.add(btnReset);
58     btnReset.addActionListener(btnListener);
59 }

```

Figure 10. NumberGrid source code(2)

The buttons share the same listener of the `ButtonListener` class, which is a named inner class.

2.2.3 Complete inner class `ButtonListener`

Your task is to complete the implementation of the `ButtonListener` class below:

```

71- private class ButtonListener implements ActionListener{
72-     @Override
73     public void actionPerformed(ActionEvent e) {
74         String button = e.getActionCommand();
75         if(button.charAt(0) >= '0' && button.charAt(0) <= '9') {
76             tfDisplay.setText(tfDisplay.getText() + button);
77         }
78         else if (button.equals("DEL")) {
79             //handles the "DEL" case
80         }
81         else {
82             //handles the "C" case
83         }
84     }
85 }
86
87 }

```

Figure 11. NumberGrid source code(3)

In the `actionPerformed()` method, we will handle the button pressed event. Since we have many sources, we need to determine which source is firing the event (which button is pressed) and handle each case accordingly (change the text of the display text field). Here, we have three cases:

- A digit button: a digit is appended to the end
- DEL button: delete the last digit
- C button: clears all digits

The code for the first case is there for reference, you need to implement it by yourself in the remaining two cases.

3 Create a graphical user interface for AIMS with Swing

Additional Requirement: From this lab, we will split the AIMS system into two applications according to the role of the user: Customer and Store Manager

- Store Manager can view and update the store (i.e: Add Book, Add CD, Add DVD).
- Customers can view the store as the store manager, but they can choose to add media to the cart. Customers can also perform some functions related to the cart.

In this lab, we will implement the application for Store Manager. The application for Customer will be implemented in the next lab.

Please put the source code in these exercises in the package **"hust.soict.dsai.aims.screen.manager"** package (for DS&AI).

3.1 View Store Screen

After starting the application for Store Manager, the user will see a view store screen as below:

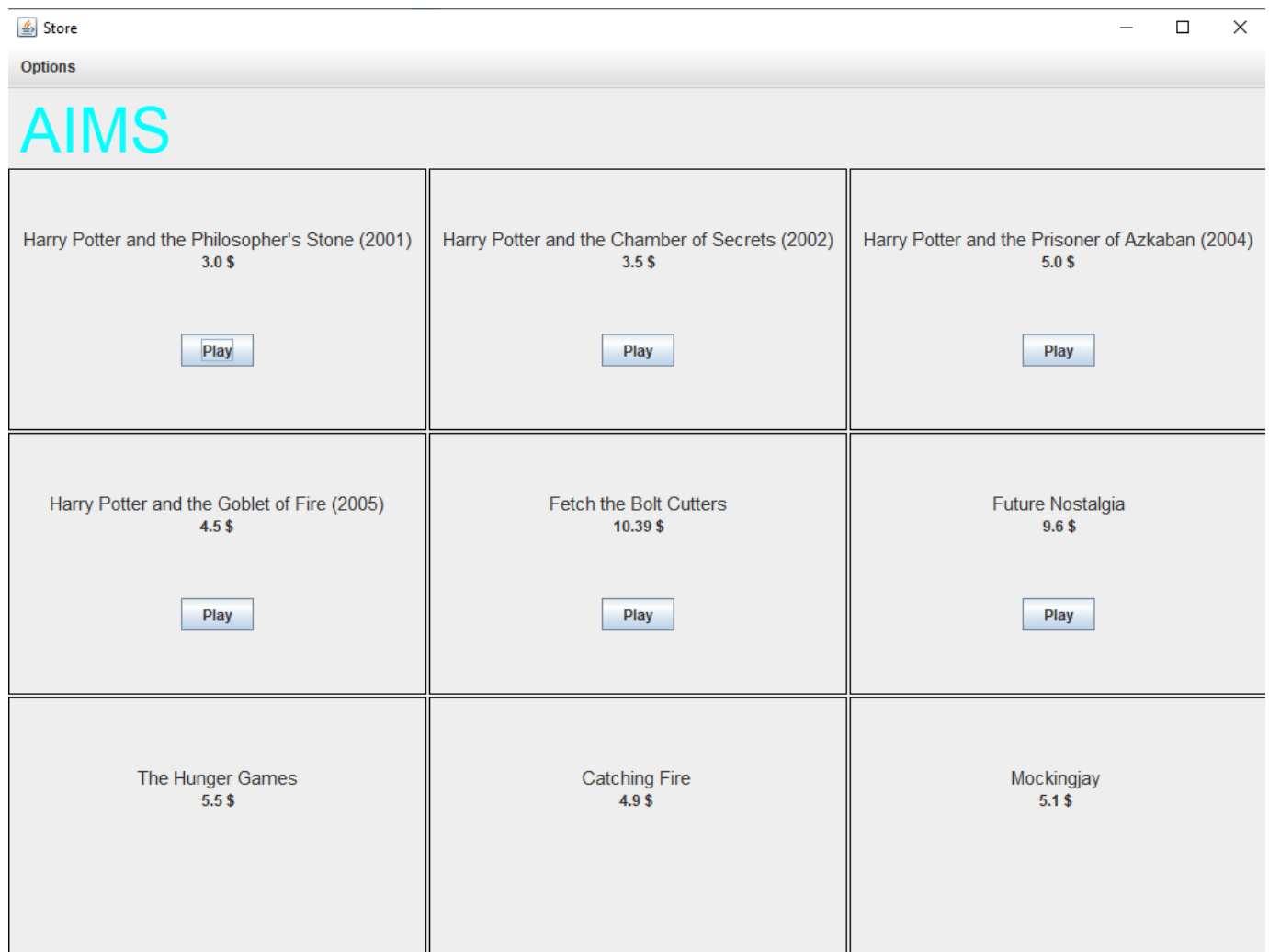


Figure 16. View Store Screen of Store Manager

For the view store screen, we will use the BorderLayout:

In the NORTH component, there will be the menu bar and the header

In the CENTER component, there will be a panel that uses the GridLayout, each cell is an item in the store.

3.1.1 Create the StoreManagerScreen class:

```
public class StoreManagerScreen{  
    private Store store;
```

Figure 17. Declaration of StoreManagerScreen class

This will be our view store screen while logging in with the role Store Manager.

Declare one attribute in the StoreManagerScreen class: Store store. This is because we need information on the items in the store to display them.

3.1.2 The NORTH component:

Create the method createNorth(), which will create our NORTH component:

```
JPanel createNorth() {  
    JPanel north = new JPanel();  
    north.setLayout(new BorderLayout(north, BorderLayout.Y_AXIS));  
    north.add(createMenuBar());  
    north.add(createHeader());  
    return north;  
}
```

Figure 18. createNorth() source code

Create the method createMenuBar():

```
JMenuBar createMenuBar() {  
    JMenu menu = new JMenu("Options");  
  
    menu.add(new JMenuItem("View store"));  
  
    JMenu smUpdateStore = new JMenu("Update Store");  
    smUpdateStore.add(new JMenuItem("Add Book"));  
    smUpdateStore.add(new JMenuItem("Add CD"));  
    smUpdateStore.add(new JMenuItem("Add DVD"));  
    menu.add(smUpdateStore);  
  
    JMenuBar menuBar = new JMenuBar();  
    menuBar.setLayout(new FlowLayout(FlowLayout.LEFT));  
    menuBar.add(menu);  
  
    return menuBar;  
}
```

Figure 19. createMenuBar() source code

The resulting menu bar will look something like this:

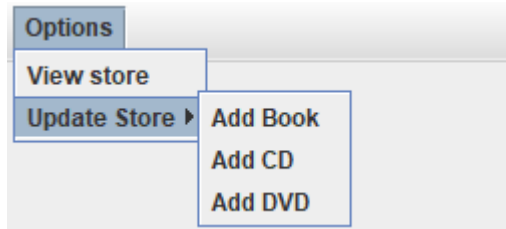


Figure 20. The resulting menu on the menu bar

Create the method `createHeader()`:

```
JPanel createHeader() {
    JPanel header = new JPanel();
    header.setLayout(new BorderLayout(header, BorderLayout.X_AXIS));

    JLabel title = new JLabel("AIMS");
    title.setFont(new Font(title.getFont().getName(), Font.PLAIN, 50));
    title.setForeground(Color.CYAN);

    header.add(Box.createRigidArea(new Dimension(10, 10)));
    header.add(title);
    header.add(Box.createHorizontalGlue());
    header.add(Box.createRigidArea(new Dimension(10, 10)));

    return header;
}
```

Figure 21. createHeader() source code

3.1.3 The CENTER component:

```
101  JPanel createCenter() {
102
103      JPanel center = new JPanel();
104      center.setLayout(new GridLayout(3, 3, 2, 2));
105
106      ArrayList<Media> mediaInStore = store.getItemsInStore();
107      for (int i = 0; i < 9; i++) {
108          MediaStore cell = new MediaStore(mediaInStore.get(i));
109          center.add(cell);
110      }
111
112
113      return center;
114  }
```

Figure 22. createCenter() source code

Here, we see that each cell is an object of the class `MediaStore`, which represents the GUI element for a Media in the Store Screen.

3.1.4 The MediaStore class:

Here, since the `MediaStore` is a GUI element, it extends the `JPanel` class. It has one attribute: `Media media`.

```

public class MediaStore extends JPanel{
    private Media media;
    public MediaStore(Media media) {
        this.media = media;
        this.setLayout(new BoxLayout(this, BoxLayout.Y_AXIS));

        JLabel title = new JLabel(media.getTitle());
        title.setFont(new Font(title.getFont().getName(), Font.PLAIN, 15));
        title.setAlignmentX(CENTER_ALIGNMENT);

        JLabel cost = new JLabel(""+media.getCost()+" $");
        cost.setAlignmentX(CENTER_ALIGNMENT);

        JPanel container = new JPanel();
        container.setLayout(new FlowLayout(FlowLayout.CENTER));

        if(media instanceof Playable) {
            JButton playButton = new JButton("Play");
            container.add(playButton);
        }

        this.add(Box.createVerticalGlue());
        this.add(title);
        this.add(cost);
        this.add(Box.createVerticalGlue());
        this.add(container);

        this.setBorder(BorderFactory.createLineBorder(Color.BLACK));
    }
}

```

Figure 23. MediaStore source code

Note how the code checks if the Media implements the Playable interface to create a “Play” button.

3.1.5 Putting it all together:

Finally, we have all the component methods to use in the constructor of StoreScreen:

```

public StoreManagerScreen(Store store) {
    this.store = store;

    Container cp = getContentPane();
    cp.setLayout(new BorderLayout());
    cp.add(createNorth(), BorderLayout.NORTH);
    cp.add(createCenter(), BorderLayout.CENTER);

    setTitle("Store");
    setSize(1024, 768);
    setLocationRelativeTo(null);
    setVisible(true);
}

```

Add main method to start the application for Store Manager.

3.2 Update Store Screen

We have successfully set up all the components for our store, but they are just static – buttons and menu items don't respond when clicked. Now, it's your task to implement the handling of the event when the user interacts with:

- The menu bar:
 - When a user clicks on one of the items of the “Update Store” menu on the menu bar (such as “Add Book”, “Add CD”, or “Add DVD”), the application should switch to an appropriate new screen for the user to add the new item. This screen should have the same menu bar as the View Store Screen, so the user can go back to view the store.
 - When the user clicks on “View Store”, the application should switch to the View Store Screen.
- The buttons on MediaHome:
 - When the user clicks on the “Play” button, the Media should be played in a dialog window. You can use JDialog here.

Note:

- For simplicity:
 - You only need to do the “Add item to store” screen, the “Remove item from store” is omitted. As shown above, there is no option of “Remove item from store” in the “Update Store” menu.
 - In the “Add item to store” Screen, you don't need to do data type validation yet.

Suggestion: You might need to create more screens for this task. Make other modifications to the source code above as needed. You should create 3 classes AddDigitalVideoDiscToStoreScreen, AddCompactDiscToStoreScreen, and AddBookToStoreScreen, which will take the necessary inputs from the user. Since these GUI classes all share part of their interface, you can apply Inheritance here and create a parent AddItemToStoreScreen class and let the above three classes inherit from it.

- Update the UML class diagram for the **AimsProject**. Update the new .astah & .png file in the **Design** directory. We can apply Release Flow here by creating a branch, e.g., **topic/update-class-diagram/aims-project/lab04**, push the diagram and its image, and then merge with master.